Hopenet: Evaluasi Cross-Dataset Zero-Shot (AFLW2000-3D dan MPIIFaceGaze)

Tanpa training / fine-tuning. Bobot pretrained Hopenet (300W-LP) langsung diuji pada:

1. **AFLW2000-3D** (hasil utama): yaw, pitch, roll.
2. **MPIIFaceGaze normalized** (pelengkap): yaw/pitch diturunkan dari 3DHead. Roll tidak dievaluasi (konstan nol).
3. **MPIIFaceGaze asli** (pelengkap): yaw/pitch/roll diturunkan dari rotation vector di `pXX.txt`.

Alur: setup -> konfigurasi -> model -> AFLW2000 -> MPII normalized -> MPII asli -> ringkasan.

Catatan metodologi (tulis di laporan):

- Margin crop dipilih pada data evaluasi itu sendiri (opsi diskret dan sedikit, tetapi tetap disebutkan). Preprocessing Hopenet (RGB, ImageNet norm, 224x224) tetap sesuai skrip resmi.
- Di MPII, tanda sumbu dikalibrasi dari tanda korelasi prediksi dengan label turunan.
- Label pose MPIIFaceGaze berasal dari estimasi fitting model wajah 3D, bukan sensor ground truth.
- Selalu bandingkan dengan baseline tebak-median.

```
from google.colab import drive
drive.mount('/content/drive')
import os, sys, glob, zipfile, itertools, random, subprocess
import numpy as np, pandas as pd, cv2, torch, torchvision
import torch.nn.functional as F
import matplotlib.pyplot as plt
from scipy.io import loadmat
from scipy.spatial.transform import Rotation as Rot
from concurrent.futures import ThreadPoolExecutor
from tqdm.auto import tqdm
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("torch:", torch.__version__, "| device:", device,
      "|", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "-")
```

```
Mounted at /content/drive
torch: 2.11.0+cu130 | device: cuda | Tesla T4
```

Konfigurasi

```
# ================= KONFIGURASI =================
OUT_DIR   = '/content/drive/MyDrive/Hopenet_Protocol1/zeroshot_eval'
WEIGHTS   = {'robust_alpha1': '/content/drive/MyDrive/HopeNet/hopenet_robust_alpha1.pkl'}
AFLW_ZIP  = '/content/drive/MyDrive/HopeNet/AFLW2000-3D.zip'
MPII_ZIP  = '/content/drive/MyDrive//Dataset/MPIIFaceGaze_normalized.zip'   # sesuaikan lokasi
ORIG_ZIP  = '/content/drive/MyDrive/Dataset/MPIIFaceGaze.zip'              # sesuaikan lokasi
IMG_SIZE  = 224                  # input Hopenet
SEED      = 0
N_BOOT    = 1000                 # resampling bootstrap untuk CI 95%
AFLW_MARGINS   = [0.0, 0.2, 0.4, 0.6]       # margin total crop dari bbox landmark (bbox x (1+margin))
AFLW_MAX_ANGLE = 99                          # protokol standar: buang |sudut| > 99 derajat
MPII_CROPS     = [0.7, 0.85, 1.0, 1.2, 1.4]  # <=1 : zoom-in (center crop), >1 : tambah padding
MPII_OUTLIER   = 45
MPII_SWEEP_N   = 2000
MPII_EVAL_N    = None            # None = semua frame (~37.600); isi mis. 10000 untuk uji cepat
ORIG_SCALES    = [2.0, 2.5, 3.0, 3.5]       # lebar crop = skala x bbox 6 landmark
ORIG_OUTLIER   = 45
ORIG_SWEEP_N   = 2000
# ===============================================
os.makedirs(OUT_DIR, exist_ok=True)
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
```

```
<torch._C.Generator at 0x7ba8b17536d0>
```

Model Hopenet

```
class Hopenet(torch.nn.Module):
    def __init__(self, num_bins=66):
        super().__init__()
        r = torchvision.models.resnet50(weights=None)
        self.conv1, self.bn1, self.relu, self.maxpool = r.conv1, r.bn1, r.relu, r.maxpool
        self.layer1, self.layer2, self.layer3, self.layer4 = r.layer1, r.layer2, r.layer3, r.layer4
        self.avgpool = torch.nn.AdaptiveAvgPool2d(1)
        self.fc_yaw = torch.nn.Linear(2048, num_bins)
        self.fc_pitch = torch.nn.Linear(2048, num_bins)
        self.fc_roll = torch.nn.Linear(2048, num_bins)
        self.fc_finetune = torch.nn.Linear(2048 + 3, 3)   # tidak dipakai; ada di snapshot asli
    def forward(self, x):
        x = self.maxpool(self.relu(self.bn1(self.conv1(x))))
        x = self.layer4(self.layer3(self.layer2(self.layer1(x))))
        x = torch.flatten(self.avgpool(x), 1)
        return self.fc_yaw(x), self.fc_pitch(x), self.fc_roll(x)

def _torch_load(path):
    try:
        return torch.load(path, map_location='cpu', weights_only=True)
    except Exception as e:
        print(f"weights_only=True gagal ({type(e).__name__}); memuat dengan weights_only=False (pastikan file tepercaya).")
        return torch.load(path, map_location='cpu', weights_only=False)

def load_hopenet(path):
    assert os.path.exists(path), f"Bobot tidak ditemukan: {path}"
    sd = _torch_load(path)
    if isinstance(sd, dict) and 'state_dict' in sd:
        sd = sd['state_dict']
    sd = {(k[7:] if k.startswith('module.') else k): v for k, v in sd.items()}
    m = Hopenet(66)
    missing, unexpected = m.load_state_dict(sd, strict=False)
    crit = [k for k in missing if not k.startswith('fc_finetune') and 'num_batches_tracked' not in k]
    assert not crit, f"Bobot penting tidak termuat: {crit[:10]}"
    print(f"kunci hilang (penting): {len(crit)} | tak terpakai: {len(unexpected)}")
    return m.to(device).eval()

MODELS = {t: load_hopenet(p) for t, p in WEIGHTS.items()}
TAG = list(MODELS)[0]            # bobot yang dievaluasi
MODEL = MODELS[TAG]
print("Model dimuat:", TAG)
```

```
kunci hilang (penting): 0 | tak terpakai: 0
Model dimuat: robust_alpha1
```

Fungsi bersama (crop, prediksi, metrik)

```
IM_MEAN = np.array([0.485, 0.456, 0.406], np.float32)
IM_STD  = np.array([0.229, 0.224, 0.225], np.float32)
IDX66   = torch.arange(66, dtype=torch.float32, device=device)

def read_img(path):
    im = cv2.imread(path)
    if im is None:
        raise IOError(f"Gambar tidak terbaca: {path}")
    return im

def crop_square(im, pts, margin, out=IMG_SIZE):
    # crop persegi dari bbox landmark (pts: 2xN) diperlebar 'margin', resize ke 'out'
    x0, y0 = pts.min(1); x1, y1 = pts.max(1)
    cx, cy = (x0 + x1) / 2, (y0 + y1) / 2
    half = max(x1 - x0, y1 - y0) * (1 + margin) / 2
    l, t, r, b = [int(round(v)) for v in (cx - half, cy - half, cx + half, cy + half)]
    h, w = im.shape[:2]
    pl, pt_, pr, pb = max(0, -l), max(0, -t), max(0, r - w), max(0, b - h)
    if max(pl, pt_, pr, pb) > 0:
        im = cv2.copyMakeBorder(im, pt_, pb, pl, pr, cv2.BORDER_CONSTANT, value=0)
        l += pl; r += pl; t += pt_; b += pt_
    return cv2.resize(im[t:b, l:r], (out, out), interpolation=cv2.INTER_AREA)

def to_input(crops):
    # BGR uint8 (N,H,W,3) -> RGB, /255, normalisasi ImageNet (sama dengan skrip uji Hopenet)
    a = np.asarray(crops)[..., ::-1].astype(np.float32) / 255.0
    a = (a - IM_MEAN) / IM_STD
    return torch.from_numpy(np.ascontiguousarray(a)).permute(0, 3, 1, 2)

@torch.no_grad()
def predict(crops, bs=128):
    # -> (N,3) derajat [yaw, pitch, roll]; decoding resmi Hopenet (ekspektasi softmax 66 bin)
    out = []
    for i in range(0, len(crops), bs):
        logits = MODEL(to_input(crops[i:i + bs]).to(device))
        ang = [(F.softmax(l, dim=1) * IDX66).sum(1) * 3 - 99 for l in logits]
        out.append(torch.stack(ang, 1).float().cpu())
    return torch.cat(out).numpy()

def predict_idx(idx, make_crop, desc, chunk=256):
    # baca + crop + prediksi per potongan (hemat RAM, tidak menyimpan seluruh crop 224x224)
    P = []
    with ThreadPoolExecutor(8) as ex:
        for s in tqdm(range(0, len(idx), chunk), desc=desc, leave=False):
            crops = list(ex.map(make_crop, idx[s:s + chunk]))
            P.append(predict(np.stack(crops)))
    return np.concatenate(P)

def boot_ci(e, n=N_BOOT, seed=SEED):
    rng = np.random.RandomState(seed)
    m = np.array([e[rng.randint(0, len(e), len(e))].mean() for _ in range(n)])
    return np.percentile(m, [2.5, 97.5])

def metric_table(P, Y, axes, tag):
    e = P - Y; ae = np.abs(e); base = np.abs(Y - np.median(Y, 0)); rows = []
    for k, a in enumerate(axes):
        lo, hi = boot_ci(ae[:, k])
        rows.append(dict(sumbu=a, MAE=ae[:, k].mean(), CI95=f'[{lo:.2f}, {hi:.2f}]',
                         MAE_baseline=base[:, k].mean(), Median=np.median(ae[:, k]),
                         RMSE=np.sqrt((e[:, k] ** 2).mean()), Bias=e[:, k].mean(),
                         Pearson=np.corrcoef(P[:, k], Y[:, k])[0, 1],
                         **{f'SR{t}(%)': (ae[:, k] < t).mean() * 100 for t in (5, 10, 15)}))
    lo, hi = boot_ci(ae.mean(1))
    rows.append(dict(sumbu='rata-rata', MAE=ae.mean(), CI95=f'[{lo:.2f}, {hi:.2f}]', MAE_baseline=base.mean(),
                     Median=np.median(ae.mean(1)), RMSE=np.sqrt((e ** 2).mean()), Bias=np.nan, Pearson=np.nan,
                     **{f'SR{t}(%)': (ae.max(1) < t).mean() * 100 for t in (5, 10, 15)}))
    tab = pd.DataFrame(rows)
    print(f"\n=== {tag}: metrik (baris rata-rata: SR = semua sumbu < ambang) ===")
    print(tab.round(3).to_string(index=False))
    tab.to_csv(f'{OUT_DIR}/{tag}_metrik.csv', index=False)
    return tab

THRESH = [5, 10, 15, 20, 30]
def tail_report(err, axes, tag, base_err=None):
    cols = {a: err[:, i] for i, a in enumerate(axes)}
    cols['rata-rata sumbu'] = err.mean(1)
    rows, rows_th = [], []
    for name, e in cols.items():
        rows.append(dict(sumbu=name, mean=e.mean(), median=np.median(e), std=e.std(),
                         p90=np.percentile(e, 90), p95=np.percentile(e, 95),
                         p99=np.percentile(e, 99), max=e.max()))
        rows_th.append(dict(sumbu=name, **{f">{t}°": (e > t).mean() * 100 for t in THRESH}))
    t1, t2 = pd.DataFrame(rows), pd.DataFrame(rows_th)
    print(f"\n=== {tag}: statistik error (derajat) ===");  print(t1.round(2).to_string(index=False))
    print(f"\n=== {tag}: persentase sampel dengan error > ambang (%) ==="); print(t2.round(2).to_string(index=False))
    t1.to_csv(f'{OUT_DIR}/{tag}_statistik.csv', index=False)
    t2.to_csv(f'{OUT_DIR}/{tag}_ambang.csv', index=False)
    plt.figure(figsize=(7, 4.5))
    xmax = np.percentile(err.mean(1), 99.5) * 1.2
    for name, e in cols.items():
        s = np.sort(e); plt.plot(s, np.arange(1, len(s) + 1) / len(s) * 100, label=name)
    if base_err is not None:
        s = np.sort(base_err.mean(1))
        plt.plot(s, np.arange(1, len(s) + 1) / len(s) * 100, 'k--', label='baseline median (rata-rata sumbu)')
        xmax = max(xmax, np.percentile(base_err.mean(1), 95))
    plt.xlim(0, xmax); plt.ylim(0, 100); plt.grid(alpha=.3)
    plt.xlabel('Error (derajat)'); plt.ylabel('% sampel dengan error <= x')
    plt.title(f'Kurva kumulatif error: {tag}'); plt.legend()
    plt.tight_layout(); plt.savefig(f'{OUT_DIR}/{tag}_cdf.png', dpi=150); plt.show()

def subject_table(P, Y, subj, axes, tag):
    # MAE per subjek, baseline global, baseline per subjek, dan MAE setelah offset per subjek dihilangkan
    err = np.abs(P - Y); base = np.abs(Y - np.median(Y, 0)); rows = []
    for s in sorted(set(subj)):
        mk = subj == s
        d = P[mk] - Y[mk]
        bs = np.abs(Y[mk] - np.median(Y[mk], 0))
        rows.append(dict(subject=s, n=int(mk.sum()), MAE=err[mk].mean(), base_global=base[mk].mean(),
                         base_per_subjek=bs.mean(), MAE_tanpa_offset=np.abs(d - d.mean(0)).mean(),
                         **{f'bias_{a}': d[:, k].mean() for k, a in enumerate(axes)}))
    t = pd.DataFrame(rows)
    avg = t.drop(columns='subject').mean(numeric_only=True); avg['subject'] = 'RATA-RATA'
    t = pd.concat([t, pd.DataFrame([avg])], ignore_index=True)
    print(f"\n=== {tag}: per subjek ===")
    print(t.round(2).to_string(index=False))
    print("Catatan: MAE_tanpa_offset dibandingkan dengan base_per_subjek, bukan base_global.")
    t.to_csv(f'{OUT_DIR}/{tag}_per_subjek.csv', index=False)
    tt = t.iloc[:-1]; xs = np.arange(len(tt)); w = 0.27
    plt.figure(figsize=(12, 4))
    plt.bar(xs - w, tt['MAE'], w, label=f'Hopenet zero-shot ({TAG})')
    plt.bar(xs, tt['base_global'], w, label='baseline median global')
    plt.bar(xs + w, tt['base_per_subjek'], w, label='baseline median per subjek')
    plt.xticks(xs, tt['subject'], rotation=45); plt.ylabel('MAE (derajat)')
    plt.title(f'{tag}'); plt.legend(); plt.tight_layout()
    plt.savefig(f'{OUT_DIR}/{tag}_per_subjek.png', dpi=150); plt.show()
    return t

def draw_axis(img, yaw, pitch, roll, size=100, thick=3):
    p, y, r = np.radians(pitch), -np.radians(yaw), np.radians(roll)
    h, w = img.shape[:2]; tdx, tdy = w / 2, h / 2
    x1 = size * (np.cos(y) * np.cos(r)) + tdx
    y1 = size * (np.cos(p) * np.sin(r) + np.cos(r) * np.sin(p) * np.sin(y)) + tdy
    x2 = size * (-np.cos(y) * np.sin(r)) + tdx
    y2 = size * (np.cos(p) * np.cos(r) - np.sin(p) * np.sin(y) * np.sin(r)) + tdy
    x3 = size * np.sin(y) + tdx
    y3 = size * (-np.cos(y) * np.sin(p)) + tdy
    c = (int(tdx), int(tdy))
    cv2.line(img, c, (int(x1), int(y1)), (0, 0, 255), thick)     # X merah
    cv2.line(img, c, (int(x2), int(y2)), (0, 255, 0), thick)     # Y hijau
    cv2.line(img, c, (int(x3), int(y3)), (255, 0, 0), thick)     # Z biru
    cv2.circle(img, c, 4, (255, 255, 255), -1)
    return img

def show_pose_examples(views, gt, pred, title, fname, with_roll=True):
    n = len(views); k = 3 if with_roll else 2
    fig, axs = plt.subplots(2, n, figsize=(3.4 * n, 8.2), squeeze=False)
    for j in range(n):
        for r_, (lab, ang) in enumerate([('Ground truth', gt[j]), ('Prediksi', pred[j])]):
            im = views[j].copy()
            draw_axis(im, ang[0], ang[1], ang[2] if with_roll else 0.0, size=im.shape[1] * 0.35)
            axs[r_, j].imshow(cv2.cvtColor(im, cv2.COLOR_BGR2RGB)); axs[r_, j].axis('off')
            if r_ == 1:
                lab = f"{lab} (MAE = {np.abs(pred[j][:k] - gt[j][:k]).mean():.1f})"
            txt = f"yaw = {ang[0]:.1f}   pitch = {ang[1]:.1f}"
            if with_roll:
                txt += f"\nroll = {ang[2]:.1f}"
            axs[r_, j].set_title(f"{lab}\n{txt}", fontsize=10, pad=10, linespacing=1.5)
    fig.suptitle(f"{title}\nSumbu: X merah, Y hijau, Z biru (arah hadap)", fontsize=12, y=0.99, linespacing=1.5)
    fig.subplots_adjust(top=0.82, bottom=0.02, left=0.01, right=0.99, hspace=0.35, wspace=0.06)
    plt.savefig(f'{OUT_DIR}/{fname}.png', dpi=150, bbox_inches='tight'); plt.show()

print("Fungsi bersama siap.")
```

```
Fungsi bersama siap.
```

Bagian A: AFLW2000-3D (hasil utama)

```
AFLW_DIR = '/content/aflw2000'
if not glob.glob(f'{AFLW_DIR}/**/image*.mat', recursive=True):
    os.makedirs(AFLW_DIR, exist_ok=True)
    with zipfile.ZipFile(AFLW_ZIP) as z:
        z.extractall(AFLW_DIR)
paths, angs, pts_l = [], [], []
for mp in sorted(glob.glob(f'{AFLW_DIR}/**/image*.mat', recursive=True)):
    jp = mp[:-4] + '.jpg'
    if not os.path.exists(jp):
        continue
    m = loadmat(mp)
    pitch, yaw, roll = np.degrees(np.array(m['Pose_Para']).reshape(-1)[:3])    # urutan asli AFLW2000: pitch, yaw, roll
    pts = (m['pt3d_68'] if 'pt3d_68' in m else m['pt2d'])[:2]
    paths.append(jp); angs.append([yaw, pitch, roll]); pts_l.append(pts.astype(np.float32))
assert paths, "File AFLW2000 tidak ditemukan. Cek AFLW_ZIP."
A = np.array(angs, np.float32)                                   # kolom: [yaw, pitch, roll]
keep_a = (np.abs(A) <= AFLW_MAX_ANGLE).all(1)
```

```
paths = [p for p, k in zip(paths, keep_a) if k]
pts_l = [p for p, k in zip(pts_l, keep_a) if k]
A = A[keep_a]
BASE_A = np.abs(A - np.median(A, 0)).mean(0)
print("Sampel dipakai:", len(A), "| dibuang (>%d derajat):" % AFLW_MAX_ANGLE, int((~keep_a).sum()))
print("std yaw/pitch/roll:", A.std(0).round(1))
print("Baseline tebak-median MAE (yaw,pitch,roll):", BASE_A.round(2))
```

```
Sampel dipakai: 1969 | dibuang (>99 derajat): 31
std yaw/pitch/roll: [35.3 18.1 20. ]
Baseline tebak-median MAE (yaw,pitch,roll): [26.13 12.5  13.46]
```

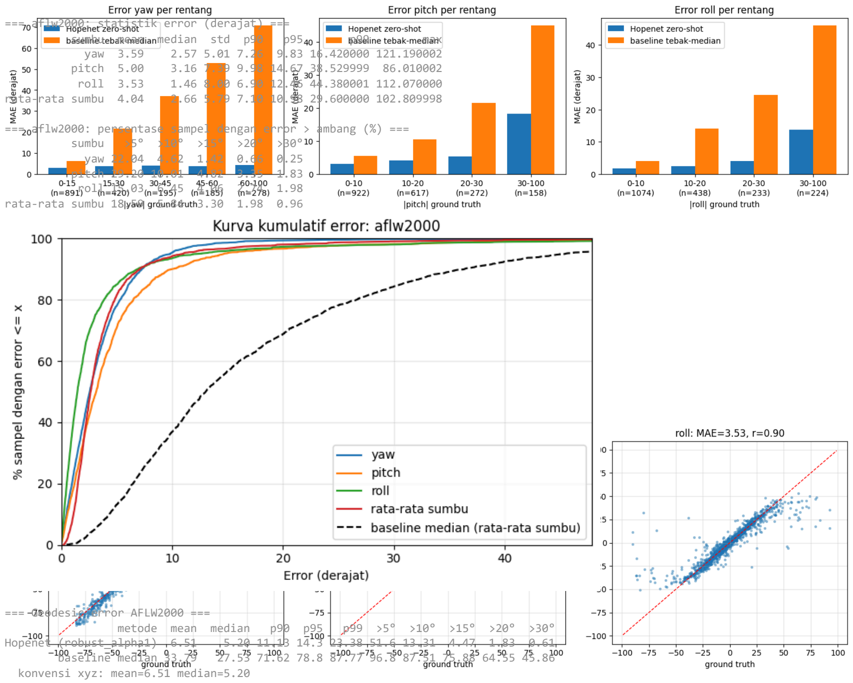
```
# Sweep margin crop (preprocessing Hopenet tetap: RGB, ImageNet norm, 224)
AX3 = ['yaw', 'pitch', 'roll']
P_BY_MARGIN, rows = {}, []
for mg in AFLW_MARGINS:
    cache = f"{OUT_DIR}/aflw_pred_hopenet_{TAG}_m{mg}_n{len(A)}.npy"
    if os.path.exists(cache) and len(np.load(cache)) == len(A):
        P = np.load(cache)
        print(f"margin {mg}: memuat prediksi dari cache")
    else:
        P = predict_idx(np.arange(len(A)), lambda i, mg=mg: crop_square(read_img(paths[i]), pts_l[i], mg),
                        f"AFLW margin {mg}")
        np.save(cache, P)
    P_BY_MARGIN[mg] = P
    mae = np.abs(P - A).mean(0)
    cor = [np.corrcoef(P[:, k], A[:, k])[0, 1] for k in range(3)]
    rows.append(dict(margin=mg, MAE_yaw=mae[0], MAE_pitch=mae[1], MAE_roll=mae[2], MAE_mean=mae.mean(),
                     corr_yaw=cor[0], corr_pitch=cor[1], corr_roll=cor[2]))
SWEEP_AFLW = pd.DataFrame(rows).sort_values('MAE_mean').reset_index(drop=True)
print(SWEEP_AFLW.round(3).to_string(index=False))
BEST_MARGIN = float(SWEEP_AFLW.loc[0, 'margin'])
print("Margin terpilih:", BEST_MARGIN)
if BEST_MARGIN in (min(AFLW_MARGINS), max(AFLW_MARGINS)):
    print("PERINGATAN: margin terpilih ada di tepi grid; pertimbangkan memperluas AFLW_MARGINS.")
if SWEEP_AFLW.loc[0, 'MAE_mean'] > 10:
    print("PERINGATAN: MAE terbaik > 10 derajat. Cek bobot/preprocessing sebelum menafsirkan MPII.")
SWEEP_AFLW.to_csv(f'{OUT_DIR}/aflw2000_sweep_margin.csv', index=False)
```

```
margin 0.0: memuat prediksi dari cache
margin 0.2: memuat prediksi dari cache
margin 0.4: memuat prediksi dari cache


 margin  MAE_yaw  MAE_pitch  MAE_roll  MAE_mean  corr_yaw  corr_pitch  corr_roll
    0.4    3.592      4.998     3.535     4.041     0.987       0.878      0.902
    0.2    3.436      5.045     3.663     4.048     0.984       0.879      0.898
    0.6    4.294      5.267     3.725     4.429     0.986       0.862      0.895
    0.0    4.389      6.549     4.489     5.142     0.982       0.837      0.880
Margin terpilih: 0.4
```

```
# Evaluasi final AFLW2000
P_A = P_BY_MARGIN[BEST_MARGIN]
err_A = np.abs(P_A - A); base_A = np.abs(A - np.median(A, 0))
print(f"=== AFLW2000 zero-shot | Hopenet {TAG} | margin={BEST_MARGIN} ===")
print("MAE yaw/pitch/roll:", err_A.mean(0).round(2), "| mean:", err_A.mean().round(2))
print("Baseline median   :", BASE_A.round(2), "| mean:", BASE_A.mean().round(2))
C = np.corrcoef(P_A.T, A.T)[:3, 3:]
print("\nKorelasi prediksi (baris) vs label (kolom):")
print(pd.DataFrame(C, index=['pred_yaw', 'pred_pitch', 'pred_roll'], columns=['gt_yaw', 'gt_pitch', 'gt_roll']).round(2))
metric_table(P_A, A, AX3, 'aflw2000')

# MAE per rentang |yaw|
rows = []
for lo, hi in [(0, 15), (15, 30), (30, 45), (45, 60), (60, 100)]:
    mk = (np.abs(A[:, 0]) >= lo) & (np.abs(A[:, 0]) < hi)
    rows.append(dict(rentang_yaw=f"{lo}-{hi}", n=int(mk.sum()), yaw=err_A[mk, 0].mean(),
                     pitch=err_A[mk, 1].mean(), roll=err_A[mk, 2].mean()))
print("\n=== MAE per rentang |yaw| ===")
print(pd.DataFrame(rows).round(2).to_string(index=False))

# MAE per rentang |nilai sumbu sendiri| dibanding baseline median
BINS = {'yaw': [(0, 15), (15, 30), (30, 45), (45, 60), (60, 100)],
        'pitch': [(0, 10), (10, 20), (20, 30), (30, 100)],
        'roll': [(0, 10), (10, 20), (20, 30), (30, 100)]}
rows = []
fig, axs = plt.subplots(1, 3, figsize=(15, 4.2)); w = 0.4
for ax_, (name, k) in zip(axs, {'yaw': 0, 'pitch': 1, 'roll': 2}.items()):
    labs, mv, bv = [], [], []
    for lo, hi in BINS[name]:
        mk = (np.abs(A[:, k]) >= lo) & (np.abs(A[:, k]) < hi)
        if mk.sum() < 5:
            continue
        labs.append(f"{lo}-{hi}\n(n={int(mk.sum())})"); mv.append(err_A[mk, k].mean()); bv.append(base_A[mk, k].mean())
        rows.append(dict(sumbu=name, rentang=f"{lo}-{hi}", n=int(mk.sum()), MAE_model=mv[-1], MAE_baseline=bv[-1]))
    xs = np.arange(len(labs))
    ax_.bar(xs - w/2, mv, w, label='Hopenet zero-shot'); ax_.bar(xs + w/2, bv, w, label='baseline tebak-median')
    ax_.set_xticks(xs); ax_.set_xticklabels(labs); ax_.set_xlabel(f'|{name}| ground truth'); ax_.set_ylabel('MAE (derajat)')
    ax_.set_title(f'Error {name} per rentang'); ax_.legend()
fig.suptitle(f'AFLW2000-3D, zero-shot Hopenet {TAG}'); plt.tight_layout()
plt.savefig(f'{OUT_DIR}/aflw2000_per_rentang.png', dpi=150); plt.show()
tab = pd.DataFrame(rows); print(tab.round(2).to_string(index=False)); tab.to_csv(f'{OUT_DIR}/aflw2000_per_rentang.csv', index=F

# Scatter prediksi vs ground truth
fig, axs = plt.subplots(1, 3, figsize=(15, 4.5))
for i, a in enumerate(AX3):
    axs[i].scatter(A[:, i], P_A[:, i], s=6, alpha=.4); axs[i].plot([-99, 99], [-99, 99], 'r--', lw=1)
    axs[i].set_title(f'{a}: MAE={err_A[:, i].mean():.2f}, r={np.corrcoef(P_A[:, i], A[:, i])[0, 1]:.2f}')
    axs[i].set_xlabel('ground truth'); axs[i].set_ylabel('prediksi'); axs[i].grid(alpha=.3)
plt.tight_layout(); plt.savefig(f'{OUT_DIR}/aflw2000_scatter.png', dpi=150); plt.show()

pd.DataFrame(dict(file=[os.path.basename(p) for p in paths], yaw_gt=A[:, 0], pitch_gt=A[:, 1], roll_gt=A[:, 2],
                  yaw_pred=P_A[:, 0], pitch_pred=P_A[:, 1], roll_pred=P_A[:, 2])
             ).to_csv(f'{OUT_DIR}/aflw2000_zeroshot_per_sample.csv', index=False)
```

```
=== AFLW2000 zero-shot | Hopenet robust_alpha1 | margin=0.4 ===
MAE yaw/pitch/roll: [3.59 5.   3.53] | mean: 4.04
Baseline median   : [26.13 12.5  13.46] | mean: 17.36
```

```
# Median, persentil, persentase error > ambang, kurva kumulatif
tail_report(err_A, AX3, 'aflw2000', base_err=base_A)

# Geodesic error rotasi
def euler_to_rot(ang, seq='xyz'):
    # ang: (N,3) [yaw, pitch, roll]. Pemetaan: x = pitch, y = yaw, z = roll (verifikasi konvensi sebelum dilaporkan)
    d = {'x': ang[:, 1], 'y': ang[:, 0], 'z': ang[:, 2]}
    return Rot.from_euler(seq, np.c_[d[seq[0]], d[seq[1]], d[seq[2]]], degrees=True)
def geodesic_deg(pred, gt, seq='xyz'):
    return (euler_to_rot(pred, seq).inv() * euler_to_rot(gt, seq)).magnitude() * 180 / np.pi
g = geodesic_deg(P_A, A); g_base = geodesic_deg(np.tile(np.median(A, 0), (len(A), 1)), A)
def stat_row(name, e):
    return dict(metode=name, mean=e.mean(), median=np.median(e), p90=np.percentile(e, 90), p95=np.percentile(e, 95),
                p99=np.percentile(e, 99), **{f">{t}°": (e > t).mean() * 100 for t in THRESH})
tab = pd.DataFrame([stat_row(f'Hopenet ({TAG})', g), stat_row('baseline median', g_base)])
print("\n=== Geodesic error AFLW2000 ===")
print(tab.round(2).to_string(index=False)); tab.to_csv(f'{OUT_DIR}/aflw2000_geodesic.csv', index=False)
for seq in ['xyz', 'zyx', 'yxz', 'zxy']:
    gg = geodesic_deg(P_A, A, seq); print(f"  konvensi {seq}: mean={gg.mean():.2f} median={np.median(gg):.2f}")
```

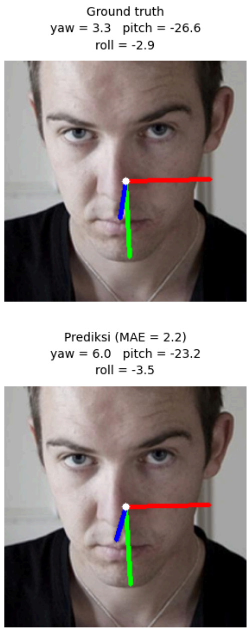


```
=== aflw2000: statistik error (derajat) ===
          sumbu  mean  median  std   p90   p95       p99         max
            yaw  3.59    2.57 5.01  7.26  9.83 16.420000 121.190002
          pitch  5.00    3.16 7.39  9.98 14.67 38.529999  86.010002
           roll  3.53    1.46 8.00  6.90 12.46 44.380001 112.070000
rata-rata sumbu  4.04    2.66 5.79  7.10 10.98 29.600000 102.809998

=== aflw2000: persentase sampel dengan error > ambang (%) ===
          sumbu   >5°  >10°  >15°  >20°  >30°
            yaw 22.04  4.62  1.42  0.66  0.25
          pitch 29.20 10.01  4.82  3.35  1.83
           roll 15.03  6.45  4.06  2.74  1.98
rata-rata sumbu 18.59  5.84  3.30  1.98  0.96

=== Geodesic error AFLW2000 ===
                 metode  mean  median   p90  p95   p99  >5°  >10°  >15°  >20°  >30°
Hopenet (robust_alpha1)  6.51    5.20 11.13 14.3 23.38 51.6 13.31  4.47  1.83  0.61
        baseline median 33.79   27.53 71.62 78.8 87.77 96.8 87.51 75.88 64.55 45.86
  konvensi xyz: mean=6.51 median=5.20
  konvensi zyx: mean=8.98 median=5.45
  konvensi yxz: mean=8.17 median=5.46
  konvensi zxy: mean=7.99 median=5.38
```

```
# Contoh satu prediksi yaw/pitch/roll (AFLW2000-3D)
IDX = None          # isi angka (mis. 123) untuk memilih gambar tertentu; None = acak
i = int(np.random.randint(len(A))) if IDX is None else int(IDX)

view = crop_square(read_img(paths[i]), pts_l[i].astype(np.float32), BEST_MARGIN, 256)
print(f"File: {os.path.basename(paths[i])} (indeks {i})")
print(pd.DataFrame({'ground truth': A[i], 'prediksi': P_A[i], 'error': np.abs(P_A[i] - A[i])},
                   index=['yaw', 'pitch', 'roll']).round(2))

show_pose_examples([view], A[[i]], P_A[[i]],
                   f"AFLW2000-3D zero-shot (Hopenet {TAG}): contoh tunggal", f"contoh_tunggal_{i}")
```

```
File: image01212.jpg (indeks 684)
       ground truth  prediksi  error
yaw        3.280000      6.04   2.76
pitch    -26.559999    -23.24   3.32
roll      -2.860000     -3.47   0.61
```

AFLW2000-3D zero-shot (Hopenet robust_alpha1): contoh tunggal
Sumbu: X merah, Y hijau, Z biru (arah hadap)



Bagian B: MPIIFaceGaze normalized (pelengkap)

Label yaw/pitch diturunkan dari kolom `3DHead` (rotation vector -> matriks rotasi -> sumbu-z kepala). Kolom 2DHead tidak dipakai.

```
DATA_DIR = '/content/data'
if not glob.glob(f'{DATA_DIR}/**/*.label', recursive=True):
    os.makedirs(DATA_DIR, exist_ok=True)
    with zipfile.ZipFile(MPII_ZIP) as z:
        z.extractall(DATA_DIR)
labels = sorted(glob.glob(f'{DATA_DIR}/**/*.label', recursive=True))
assert labels, "File .label tidak ditemukan. Cek MPII_ZIP."
label_dir = os.path.dirname(labels[0])
IMG_ROOT = os.path.join(os.path.dirname(label_dir), 'Image')
if not os.path.isdir(IMG_ROOT):
    found = glob.glob(f'{DATA_DIR}/**/Image', recursive=True)
    assert found, "Folder 'Image' tidak ditemukan."
    IMG_ROOT = found[0]
print(len(labels), "file label | IMG_ROOT:", IMG_ROOT)
subj_l, face_l, rv_l = [], [], []
for lp in labels:
    sid = os.path.splitext(os.path.basename(lp))[0]
```

```
    df = pd.read_csv(lp, sep=r'\s+')
    rv = df['3DHead'].astype(str).str.split(',', expand=True).astype(float).values
    subj_l += [sid] * len(df); face_l += df['Face'].tolist(); rv_l.append(rv)
SUBJ_ALL = np.array(subj_l); RV = np.concatenate(rv_l).astype(np.float64)
assert not np.isnan(RV).any(), "3DHead mengandung NaN"
Rm = Rot.from_rotvec(RV).as_matrix(); vz = Rm[:, :, 2]
yaw_n   = np.degrees(np.arctan2(vz[:, 0], vz[:, 2]))
pitch_n = np.degrees(np.arcsin(np.clip(-vz[:, 1], -1, 1)))
print("Total sampel:", len(RV), "| std yaw/pitch:", np.array([yaw_n.std(), pitch_n.std()]).round(2))
keep = (np.abs(yaw_n) < MPII_OUTLIER) & (np.abs(pitch_n) < MPII_OUTLIER)
print("Dibuang (outlier > %d derajat):" % MPII_OUTLIER, int((~keep).sum()))
ids = np.where(keep)[0]
if MPII_EVAL_N and MPII_EVAL_N < len(ids):
    ids = np.sort(np.random.RandomState(SEED).choice(ids, MPII_EVAL_N, replace=False))
SUBJ = SUBJ_ALL[ids]
Y = np.stack([yaw_n[ids], pitch_n[ids]], 1).astype(np.float32)      # [yaw, pitch]
FACES = [face_l[i] for i in ids]
BASE_M = np.abs(Y - np.median(Y, 0)).mean(0)
print("N dipakai:", len(Y), "| Baseline median MAE (yaw, pitch):", BASE_M.round(2))
```

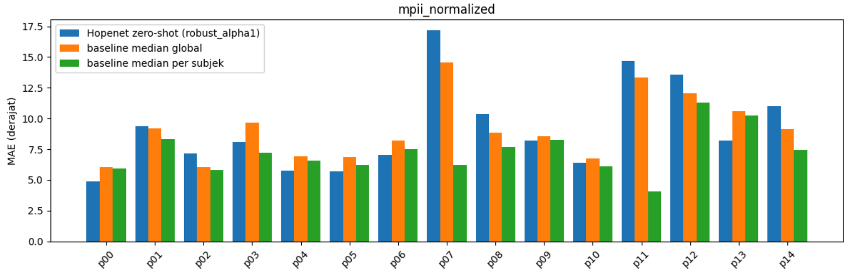
```
15 file label | IMG_ROOT: /content/data/MPIIFaceGaze_normalized/Image
Total sampel: 37667 | std yaw/pitch: [11.51 12.02]
Dibuang (outlier > 45 derajat): 16
N dipakai: 37651 | Baseline median MAE (yaw, pitch): [8.49 9.36]
```

```
def crop_mpii(im, f):
    h, w = im.shape[:2]
    if f <= 1.0:
        s = int(min(h, w) * f); y0, x0 = (h - s) // 2, (w - s) // 2
        im = im[y0:y0 + s, x0:x0 + s]
    else:
        p = int((f - 1) * h / 2)
        im = cv2.copyMakeBorder(im, p, p, p, p, cv2.BORDER_REPLICATE)
    return cv2.resize(im, (IMG_SIZE, IMG_SIZE), interpolation=cv2.INTER_AREA)

# Sweep crop + kalibrasi tanda pada subset acak
rng = np.random.RandomState(1)
sub = rng.choice(len(Y), size=min(MPII_SWEEP_N, len(Y)), replace=False)
with ThreadPoolExecutor(8) as ex:
    raw = list(ex.map(lambda i: read_img(os.path.join(IMG_ROOT, FACES[i])), sub))
gy, gp = Y[sub, 0], Y[sub, 1]
rows = []
for f in MPII_CROPS:
    P = predict(np.stack([crop_mpii(r, f) for r in raw]))
    cy = np.corrcoef(P[:, 0], gy)[0, 1]; cp = np.corrcoef(P[:, 1], gp)[0, 1]
    rows.append(dict(crop=f, corr_yaw=cy, corr_pitch=cp,
                     MAE_yaw=np.abs(P[:, 0] - np.sign(cy) * gy).mean(),
                     MAE_pitch=np.abs(P[:, 1] - np.sign(cp) * gp).mean(), abs_corr=(abs(cy) + abs(cp)) / 2))
SWEEP_M = pd.DataFrame(rows).sort_values('abs_corr', ascending=False).reset_index(drop=True)
top = SWEEP_M.iloc[0]
MPII_CFG = dict(crop=float(top['crop']), sy=int(np.sign(top['corr_yaw']) or 1), sp=int(np.sign(top['corr_pitch']) or 1))
print(SWEEP_M.round(3).to_string(index=False)); print("Terpilih:", MPII_CFG)
SWEEP_M.to_csv(f'{OUT_DIR}/mpii_sweep_crop.csv', index=False)
del raw
```

```
 crop  corr_yaw  corr_pitch  MAE_yaw  MAE_pitch  abs_corr
 0.70     0.400      -0.358    7.752     10.155     0.379
 0.85     0.389      -0.322    7.844     10.100     0.356
 1.00     0.375      -0.269    8.439     11.105     0.322
 1.20     0.341      -0.252    9.726     12.173     0.297
 1.40     0.284      -0.228   11.990     13.759     0.256
Terpilih: {'crop': 0.7, 'sy': 1, 'sp': -1}
```

```
# Evaluasi final MPII normalized
cfg = MPII_CFG
cache = f"{OUT_DIR}/mpii_norm_pred_hopenet_{TAG}_crop{cfg['crop']}_n{len(Y)}.npy"
if os.path.exists(cache):
    P_M = np.load(cache)
else:
    P_M = predict_idx(np.arange(len(Y)), lambda i: crop_mpii(read_img(os.path.join(IMG_ROOT, FACES[i])), cfg['crop']),
                      "MPII normalized")[:, :2]
    np.save(cache, P_M)
Yc = Y * np.array([cfg['sy'], cfg['sp']], np.float32)       # label dengan tanda terkalibrasi
err_M = np.abs(P_M - Yc); base_M = np.abs(Yc - np.median(Yc, 0))
print(f"=== MPII normalized zero-shot | Hopenet {TAG} | {cfg} ===")
print("MAE yaw/pitch:", err_M.mean(0).round(2), "| mean:", err_M.mean().round(2))
print("Baseline median:", base_M.mean(0).round(2), "| mean:", base_M.mean().round(2))
print("Rata-rata bias (pred - label):", (P_M - Yc).mean(0).round(2))
print("Std prediksi vs std label:", P_M.std(0).round(2), Yc.std(0).round(2))
metric_table(P_M, Yc, ['yaw', 'pitch'], 'mpii_normalized')
t_M = subject_table(P_M, Yc, SUBJ, ['yaw', 'pitch'], 'mpii_normalized')
tail_report(err_M, ['yaw', 'pitch'], 'mpii_normalized', base_err=base_M)
pd.DataFrame(dict(subject=SUBJ, face=FACES, yaw_gt=Yc[:, 0], pitch_gt=Yc[:, 1],
                  yaw_pred=P_M[:, 0], pitch_pred=P_M[:, 1])).to_csv(f'{OUT_DIR}/mpii_normalized_per_sample.csv', index=False)
```

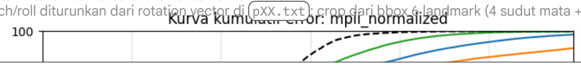
```
=== MPII normalized zero-shot | Hopenet robust_alpha1 | {'crop': 0.7, 'sy': 1, 'sp': -1} ===
MAE yaw/pitch: [ 7.52 10.35] | mean: 8.93
Baseline median: [8.49 9.36] | mean: 8.92
Rata-rata bias (pred - label): [-1.06 -7.26]
Std prediksi vs std label: [6.04 7.52] [11.46 12.  ]

=== mpii_normalized: metrik (baris rata-rata: SR = semua sumbu < ambang) ===
    sumbu    MAE           CI95  MAE_baseline  Median   RMSE   Bias  Pearson  SR5(%)  SR10(%)  SR15(%)
      yaw  7.524   [7.44, 7.60]         8.487   4.715 10.595 -1.057    0.410  51.972   73.031   83.217
    pitch 10.345 [10.26, 10.44]         9.356   7.761 13.796 -7.262    0.349  34.945   59.778   75.942
rata-rata  8.935   [8.88, 8.99]         8.922   8.051 12.300    NaN      NaN  14.411   39.136   60.330

=== mpii_normalized: per subjek ===
  subject       n   MAE  base_global  base_per_subjek  MAE_tanpa_offset  bias_yaw  bias_pitch
      p00 2926.00  4.88         6.03             5.94              4.63      1.60       -2.12
      p01 2904.00  9.38         9.18             8.30              8.56     -2.02       -7.04
      p02 2916.00  7.16         6.02             5.79              5.01      0.69      -10.69
      p03 2929.00  8.08         9.68             7.24              6.98      1.76       -4.88
      p04 2860.00  5.75         6.90             6.54              5.64      0.82       -3.09
      p05 2870.00  5.67         6.85             6.23              5.36     -3.13       -0.77
      p06 2877.00  7.05         8.17             7.49              6.11      0.68       -5.37
      p07 2842.00 17.18        14.53             6.21              8.47      1.44      -25.48
      p08 2755.00 10.35         8.83             7.69              5.38      0.90      -15.72
      p09 2719.00  8.20         8.57             8.28              8.05     -0.52        1.60
      p10 2193.00  6.36         6.76             6.13              4.59      1.11       -6.70
      p11 2262.00 14.70        13.36             4.05              5.88    -17.94       -7.74
      p12 1600.00 13.58        12.06            11.30             12.25     -2.34       -8.97
      p13 1498.00  8.20        10.57            10.26              8.06      0.43       -1.73
      p14 1500.00 10.98         9.13             7.44              7.66     -3.35       -9.52
RATA-RATA 2510.07  9.17         9.11             7.26              6.84     -1.32       -7.21
Catatan: MAE_tanpa_offset dibandingkan dengan base_per_subjek, bukan base_global.
```



```
=== mpii_normalized: statistik error (derajat) ===
          sumbu  mean  median  std       p90       p95       p99       max
            yaw  7.52    4.71 7.46 19.290001 23.209999 30.129999 60.689999
          pitch 10.35    7.76 9.13 22.920000 30.580000 39.180000 62.509998
rata-rata sumbu  8.93    8.05 5.24 16.450001 18.940001 23.200001 50.209999

=== mpii_normalized: persentase sampel dengan error > ambang (%) ===
          sumbu   >5°  >10°  >15°  >20°  >30°
            yaw [illegible]
          pitch 65.06 40.22 24.06 13.55  5.37
rata-rata sumbu 73.83 36.76 13.85  3.48  0.06
```

Bagian C: MPIIFaceGaze asli, tanpa normalisasi (pelengkap)

Yaw/pitch/roll diturunkan dari rotation vector di `pXX.txt`; crop dari bbox 6 landmark (4 sudut mata + 2 sudut mulut). Roll adalah pseudo-label.



```
ORIG_DIR = '/content/mpii_orig'
if not glob.glob(f'{ORIG_DIR}/**/p00.txt', recursive=True):
    os.makedirs(ORIG_DIR, exist_ok=True)
    with zipfile.ZipFile(ORIG_ZIP) as z:
        z.extractall(ORIG_DIR)
txts = sorted(glob.glob(f'{ORIG_DIR}/**/p[0-9][0-9].txt', recursive=True))
assert txts, "File pxx.txt tidak ditemukan. Cek ORIG_ZIP."
print(len(txts), "file anotasi | contoh baris:", open(txts[0]).readline()[:160])
```

```
def resolve(pdir, rel):
    for base in (pdir, os.path.dirname(pdir)):
        f = os.path.join(base, rel)
        if os.path.exists(f):
            return f
    return os.path.join(pdir, rel)

o_subj, o_path, o_pts, o_rv = [], [], [], []
for tp in txts:
    sid, pdir = os.path.splitext(os.path.basename(tp))[0], os.path.dirname(tp)
    for line in open(tp):
        p = line.split()
        if len(p) < 21:
            continue
        o_subj.append(sid); o_path.append(resolve(pdir, p[0]))
        o_pts.append(np.array(p[3:15], np.float32).reshape(6, 2))     # kolom 3-14: 6 landmark
        o_rv.append(np.array(p[15:18], np.float64))                   # kolom 15-17: rotation vector
assert os.path.exists(o_path[0]), f"Gambar tidak ditemukan: {o_path[0]} (cek struktur folder)"
print("Total baris:", len(o_path))

Ro = Rot.from_rotvec(np.stack(o_rv)).as_matrix()
if np.median(Ro[:, 2, 2]) < 0:
    print("Sumbu-z kepala menghadap kamera -> dibalik 180 derajat terhadap sumbu-y.")
    Ro = Ro @ np.diag([-1.0, 1.0, -1.0])
vz = Ro[:, :, 2]
yaw_o   = np.degrees(np.arctan2(vz[:, 0], vz[:, 2]))
pitch_o = np.degrees(np.arcsin(np.clip(-vz[:, 1], -1, 1)))
roll_o  = np.degrees(np.arctan2(Ro[:, 1, 0], Ro[:, 0, 0]))
for n, a in [('yaw', yaw_o), ('pitch', pitch_o), ('roll', roll_o)]:
    print(f"{n:5s} median={np.median(a):6.1f} std={a.std():5.1f} persentil 0.1/99.9={np.percentile(a, [0.1, 99.9]).round(1)}")
keep_o = (np.abs(yaw_o) < ORIG_OUTLIER) & (np.abs(pitch_o) < ORIG_OUTLIER) & (np.abs(roll_o) < ORIG_OUTLIER)
print("Dibuang (outlier):", int((~keep_o).sum()))
ids_o = np.where(keep_o)[0]
if MPII_EVAL_N and MPII_EVAL_N < len(ids_o):
    ids_o = np.sort(np.random.RandomState(SEED).choice(ids_o, MPII_EVAL_N, replace=False))
SUBJ_O = np.array(o_subj)[ids_o]
PATH_O = [o_path[i] for i in ids_o]
PTS_O  = [o_pts[i] for i in ids_o]
YO     = np.stack([yaw_o, pitch_o, roll_o], 1)[ids_o].astype(np.float32)
assert not np.isnan(YO).any()
print("N dipakai:", len(YO), "| baseline median MAE (yaw,pitch,roll):", np.abs(YO - np.median(YO, 0)).mean(0).round(2))
```

```
15 file anotasi | contoh baris: day01/0005.jpg 476 758 594 366 637 365 719 366 762 368 626 495 726 496 -0.232179 0.055685 0.0182
Total baris: 37667
yaw   median=   1.5 std= 11.7 persentil 0.1/99.9=[-30.2  37.9]
pitch median=  -5.9 std= 12.1 persentil 0.1/99.9=[-43.8  32.4]
roll  median=  -0.2 std=  4.6 persentil 0.1/99.9=[-21.4  25.9]
Dibuang (outlier): 39
N dipakai: 37628 | baseline median MAE (yaw,pitch,roll): [8.49 9.44 3.21]
```

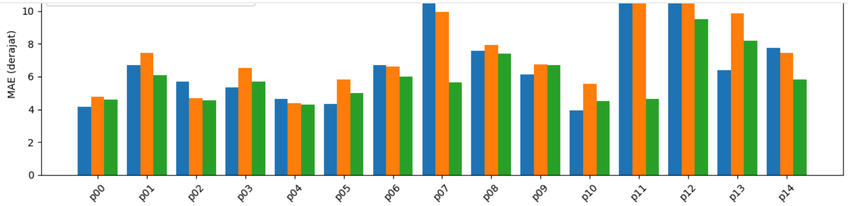
```
def crop_orig_fn(s):
    return lambda i: crop_square(read_img(PATH_O[i]), PTS_O[i].T, s - 1)    # margin = skala - 1

rng = np.random.RandomState(1)
sub = np.sort(rng.choice(len(YO), size=min(ORIG_SWEEP_N, len(YO)), replace=False))
rows = []
for s in ORIG_SCALES:
    P = predict_idx(sub, crop_orig_fn(s), f"sweep skala {s}")
    cor = [np.corrcoef(P[:, k], YO[sub, k])[0, 1] for k in range(3)]
    rows.append(dict(skala=s, corr_yaw=cor[0], corr_pitch=cor[1], corr_roll=cor[2], abs_corr=np.mean(np.abs(cor))))
SWEEP_O = pd.DataFrame(rows).sort_values('abs_corr', ascending=False).reset_index(drop=True)
top = SWEEP_O.iloc[0]
ORIG_CFG = dict(scale=float(top['skala']),
                signs=np.array([np.sign(top['corr_yaw']) or 1, np.sign(top['corr_pitch']) or 1,
                                np.sign(top['corr_roll']) or 1], np.float32))
print(SWEEP_O.round(3).to_string(index=False)); print("Terpilih:", ORIG_CFG)
SWEEP_O.to_csv(f'{OUT_DIR}/mpii_orig_sweep_skala.csv', index=False)
```

```
# Evaluasi final MPII asli
cfg = ORIG_CFG
cache = f"{OUT_DIR}/mpii_orig_pred_hopenet_{TAG}_s{cfg['scale']}_n{len(YO)}.npy"
if os.path.exists(cache):
    P_O = np.load(cache)
else:
    P_O = predict_idx(np.arange(len(YO)), crop_orig_fn(cfg['scale']), "MPII asli")
    np.save(cache, P_O)
YOc = YO * cfg['signs']
err_O = np.abs(P_O - YOc); base_O = np.abs(YOc - np.median(YOc, 0))
print(f"=== MPII ASLI zero-shot | Hopenet {TAG} | skala={cfg['scale']} | tanda={cfg['signs']} ===")
tab_ax = pd.DataFrame({'MAE_model': err_O.mean(0), 'MAE_baseline': base_O.mean(0),
                       'korelasi': [np.corrcoef(P_O[:, k], YOc[:, k])[0, 1] for k in range(3)],
                       'bias(pred-label)': (P_O - YOc).mean(0)}, index=AX3).round(2)
print(tab_ax.to_string())
print("mean(yaw,pitch): model =", err_O[:, :2].mean().round(2), "| baseline global =", base_O[:, :2].mean().round(2))
print("Catatan: roll adalah pseudo-label turunan dari rotation vector, bukan label asli dataset.")
metric_table(P_O, YOc, AX3, 'mpii_orig')
t_O = subject_table(P_O, YOc, SUBJ_O, AX3, 'mpii_orig')
tail_report(err_O, AX3, 'mpii_orig', base_err=base_O)
pd.DataFrame(dict(subject=SUBJ_O, path=PATH_O, yaw_gt=YOc[:, 0], pitch_gt=YOc[:, 1], roll_gt=YOc[:, 2],
                  yaw_pred=P_O[:, 0], pitch_pred=P_O[:, 1], roll_pred=P_O[:, 2])
             ).to_csv(f'{OUT_DIR}/mpii_orig_per_sample.csv', index=False)
```

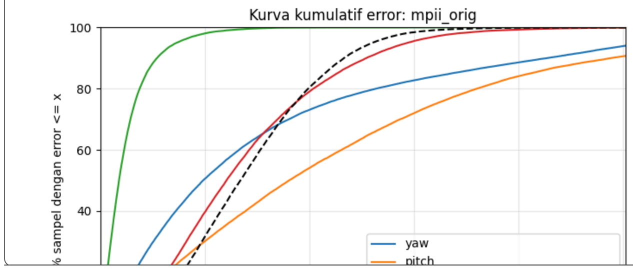
```
=== MPII ASLI zero-shot | Hopenet robust_alpha1 | skala=2.0 | tanda=[ 1. -1.  1.] ===
       MAE_model  MAE_baseline  korelasi  bias(pred-label)
yaw         8.00          8.49      0.32             -2.74
pitch      11.29          9.44      0.30             -8.28
roll        1.25          3.21      0.93              0.41
mean(yaw,pitch): model = 9.65 | baseline global = 8.97
Catatan: roll adalah pseudo-label turunan dari rotation vector, bukan label asli dataset.

=== mpii_orig: metrik (baris rata-rata: SR = semua sumbu < ambang) ===
    sumbu    MAE            CI95  MAE_baseline  Median   RMSE   Bias  Pearson  SR5(%)  SR10(%)  SR15(%)
      yaw  8.002   [7.92, 8.09]         8.494   4.921 11.564 -2.735    0.323  50.494   73.065   82.694
    pitch 11.290 [11.20, 11.40]         9.441   9.028 14.593 -8.282    0.300  30.113   54.053   71.957
     roll  1.245   [1.23, 1.26]         3.210   0.900  1.754  0.408    0.933  98.031   99.971   99.995
rata-rata  6.846   [6.81, 6.88]         7.048   6.039 10.798    NaN      NaN  14.417   38.304   58.390

=== mpii_orig: per subjek ===
```

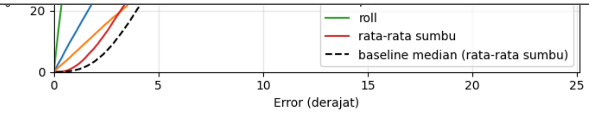
Contoh visual acak, terbaik, terburuk

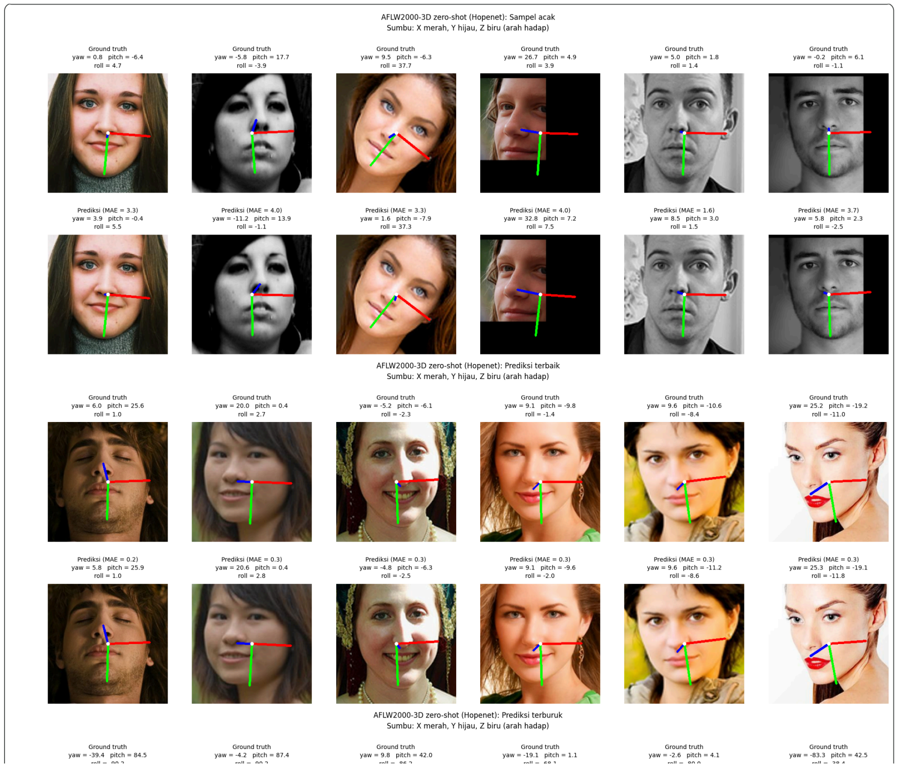
```python
N_SHOW = 6
rs = np.random.RandomState(0)

# AFLW2000
e_a = err_A.mean(1)
def view_aflw(i): return crop_square(read_img(paths[i]), pts_l[i].astype(np.float32), BEST_MARGIN, 256)
sets = {'acak': rs.choice(len(A), N_SHOW, replace=False), 'terbaik': np.argsort(e_a)[:N_SHOW], 'terburuk': np.argsort(-e_a)[:N_S
titles = {'acak': 'Sampel acak', 'terbaik': 'Prediksi terbaik', 'terburuk': 'Prediksi terburuk'}
for key, idx in sets.items():
    show_pose_examples([view_aflw(i) for i in idx], A[idx], P_A[idx], f"AFLW2000-3D zero-shot (Hopenet): {titles[key]}",
                       f"contoh_aflw2000_{key}")

# MPII normalized (roll tidak ditampilkan)
e_m = err_M.mean(1)
def view_mpii(i): return cv2.resize(read_img(os.path.join(IMG_ROOT, FACES[i])), (256, 256), interpolation=cv2.INTER_AREA)
z = lambda a: np.c_[a, np.zeros(len(a))]
sets = {'acak': rs.choice(len(Y), N_SHOW, replace=False), 'terbaik': np.argsort(e_m)[:N_SHOW], 'terburuk': np.argsort(-e_m)[:N_S
for key, idx in sets.items():
    show_pose_examples([view_mpii(i) for i in idx], z(Yc[idx]), z(P_M[idx]),
                       f"MPII normalized zero-shot (Hopenet): {titles[key]}", f"contoh_mpii_normalized_{key}", with_roll=False)
```



```
=== mpii_orig: statistik error (derajat) ===
          sumbu  mean  median  std       p90       p95       p99       max
            yaw  8.00    4.92 8.35 21.400000 26.230000 34.750000 93.120003
          pitch 11.29    9.03 9.25 24.440001 30.379999 39.950001 57.320000
           roll  1.25    0.90 1.23  2.720000  3.660000  6.000000 33.310001
rata-rata sumbu  6.85    6.04 4.16 12.730000 14.710000 18.920000 43.980000

=== mpii_orig: persentase sampel dengan error > ambang (%) ===
          sumbu   >5°  >10°  >15°  >20°  >30°
            yaw 49.51 26.93 17.31 11.45  2.69
          pitch 69.89 45.95 28.04 15.96  5.27
           roll  1.97  0.03  0.01  0.00  0.00
rata-rata sumbu 60.45 21.00  4.46  0.73  0.01
```






AFLW2000-3D zero-shot (Hopenet): Sampel acak
Sumbu: X merah, Y hijau, Z biru (arah hadap)
Ground truth
yaw = 0.8 pitch = -6.4
roll = 4.7
Ground truth
yaw = -5.8 pitch = 17.7
roll = -3.9
Ground truth
yaw = 9.5 pitch = -6.3
roll = 37.7
Ground truth
yaw = 26.7 pitch = 4.9
roll = 3.9
Ground truth
yaw = 5.0 pitch = 1.8
roll = 1.4
Ground truth
yaw = -0.2 pitch = 6.1
roll = -1.1
Prediksi (MAE = 3.3)
yaw = 3.9 pitch = -0.4
roll = 5.5
Prediksi (MAE = 4.0)
yaw = -11.2 pitch = 13.9
roll = -1.1
Prediksi (MAE = 3.3)
yaw = 1.6 pitch = -7.9
roll = 37.3
Prediksi (MAE = 4.0)
yaw = 32.8 pitch = 7.2
roll = 7.5
Prediksi (MAE = 1.6)
yaw = 8.5 pitch = 3.0
roll = 1.5
Prediksi (MAE = 3.7)
yaw = 5.8 pitch = 2.3
roll = -2.5
AFLW2000-3D zero-shot (Hopenet): Prediksi terbaik
Sumbu: X merah, Y hijau, Z biru (arah hadap)
Ground truth
yaw = 6.0 pitch = 25.6
roll = 1.0
Ground truth
yaw = 20.0 pitch = 0.4
roll = 2.7
Ground truth
yaw = -5.2 pitch = -6.1
roll = -2.3
Ground truth
yaw = 9.1 pitch = -9.8
roll = -1.4
Ground truth
yaw = 9.6 pitch = -10.6
roll = -8.4
Ground truth
yaw = 25.2 pitch = -19.2
roll = -11.0
Prediksi (MAE = 0.2)
yaw = 5.8 pitch = 25.9
roll = 1.0
Prediksi (MAE = 0.3)
yaw = 20.6 pitch = 0.4
roll = 2.8
Prediksi (MAE = 0.3)
yaw = -4.8 pitch = -6.3
roll = -2.5
Prediksi (MAE = 0.3)
yaw = 9.1 pitch = -9.6
roll = -2.0
Prediksi (MAE = 0.3)
yaw = 9.6 pitch = -11.2
roll = -8.6
Prediksi (MAE = 0.3)
yaw = 25.3 pitch = -19.1
roll = -11.8
AFLW2000-3D zero-shot (Hopenet): Prediksi terburuk
Sumbu: X merah, Y hijau, Z biru (arah hadap)
Ground truth
yaw = -39.4 pitch = 84.5
Ground truth
yaw = -4.2 pitch = 87.4
Ground truth
yaw = 9.8 pitch = 42.0
Ground truth
yaw = -19.1 pitch = 1.1
Ground truth
yaw = -2.6 pitch = 4.1
Ground truth
yaw = -83.3 pitch = 42.5